next digital

Orquestando tareas con Airflow:

Tus datos en piloto automático

Sesión 2: Siguientes pasos con Airflow



Vanessa Pradas Fernández | Data Engineer

00_
¿De dónde vengo?



+140 especialistas

ÁREAS

- Cloud
- Data & AI
- Analítica Avanzada
- Desarrollo de Software

SECTORES













Azure

amazon web services

dbt

SirviendoCódigo.com

www.youtube.com/@SirviendoCódigo



Índice

1. En la sesión anterior...
2. Objetivos
3. Conceptos avanzados
4. Buenas prácticas
5. Manos a la obra

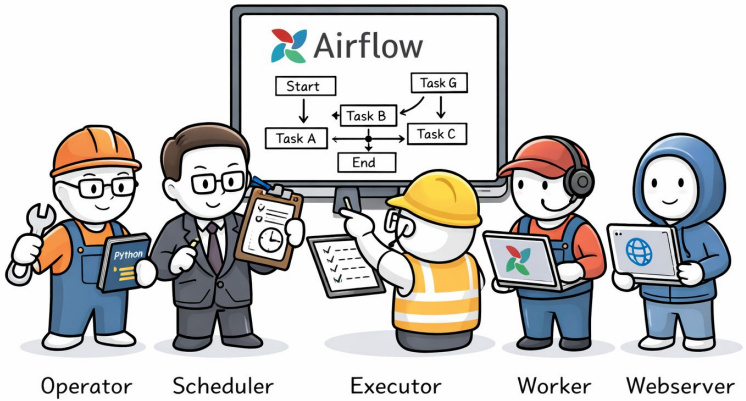




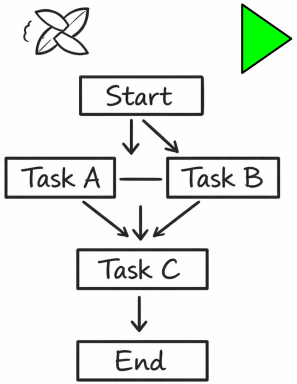
01_
En la sesión anterior...

En la sesión anterior...

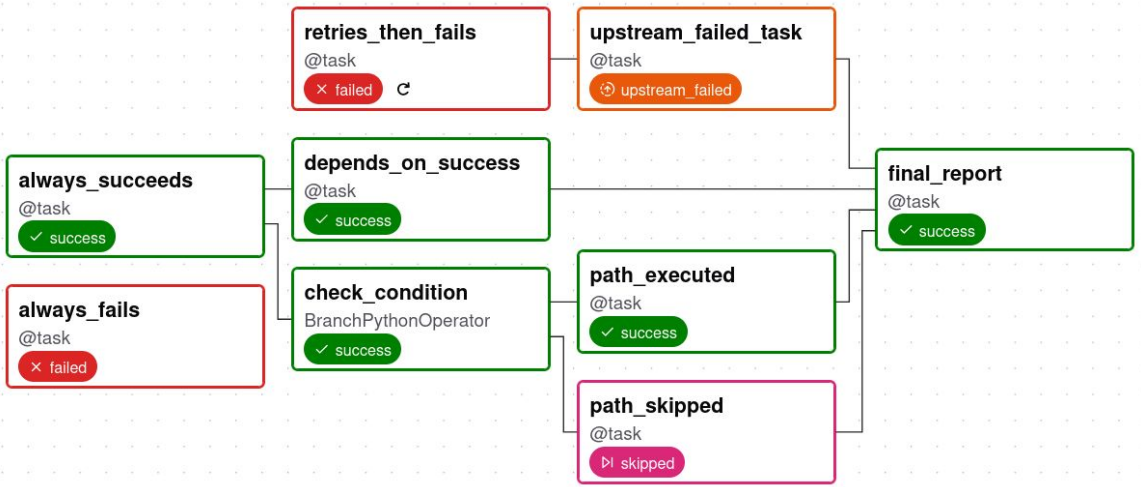
Entendimos qué es Airflow y sus componentes principales



Creamos y ejecutamos un DAG sencillo desde la interfaz de Airflow

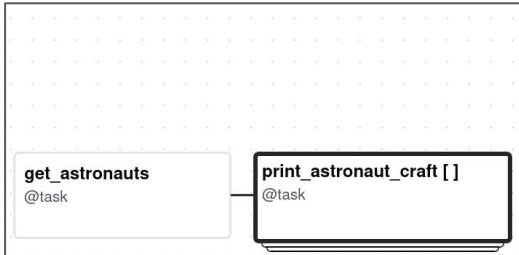


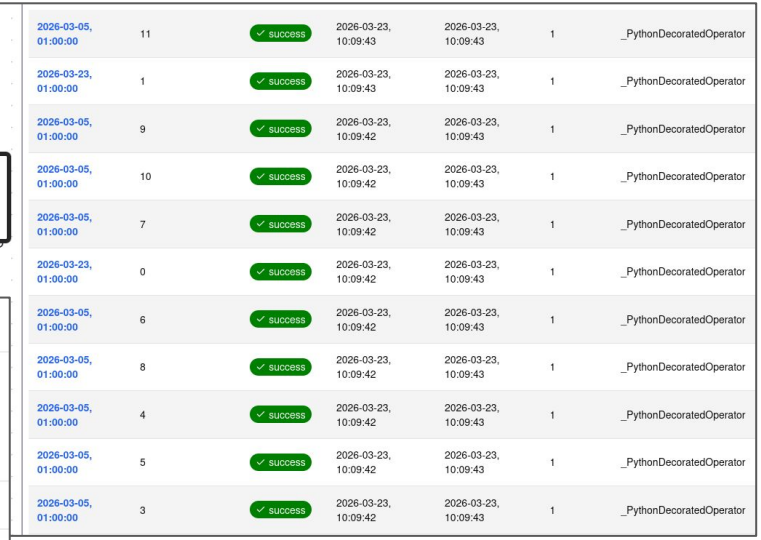
DAG de estados



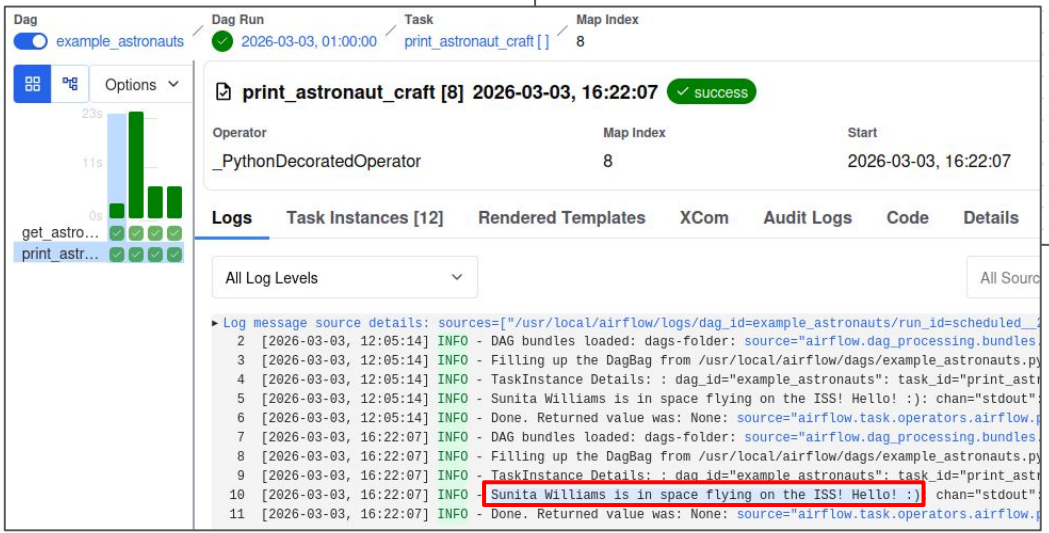
Color	Significado
🟢 Verde	La tarea terminó correctamente.
🔴 Rojo	La tarea lanzó una excepción.
🟡 Amarillo	Falló pero se va a reintentar.
🟠 Naranja	No se ejecutó porque una anterior falló.
🩷 Rosa	Se saltó (por branching o condición).
🟣 Morado	Se está ejecutando ahora mismo.

DAG de astronautas





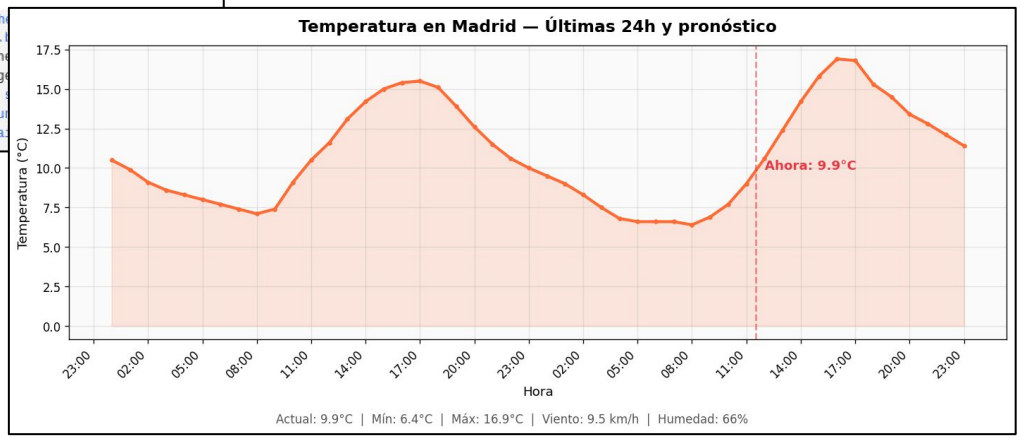
2026-03-05, 01:00:00	11	success	2026-03-23, 10:09:43	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-23, 01:00:00	1	success	2026-03-23, 10:09:43	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	9	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	10	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	7	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-23, 01:00:00	0	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	6	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	8	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	4	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	5	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator
2026-03-05, 01:00:00	3	success	2026-03-23, 10:09:42	2026-03-23, 10:09:43	1	_PythonDecoratedOperator



DAG de temperatura

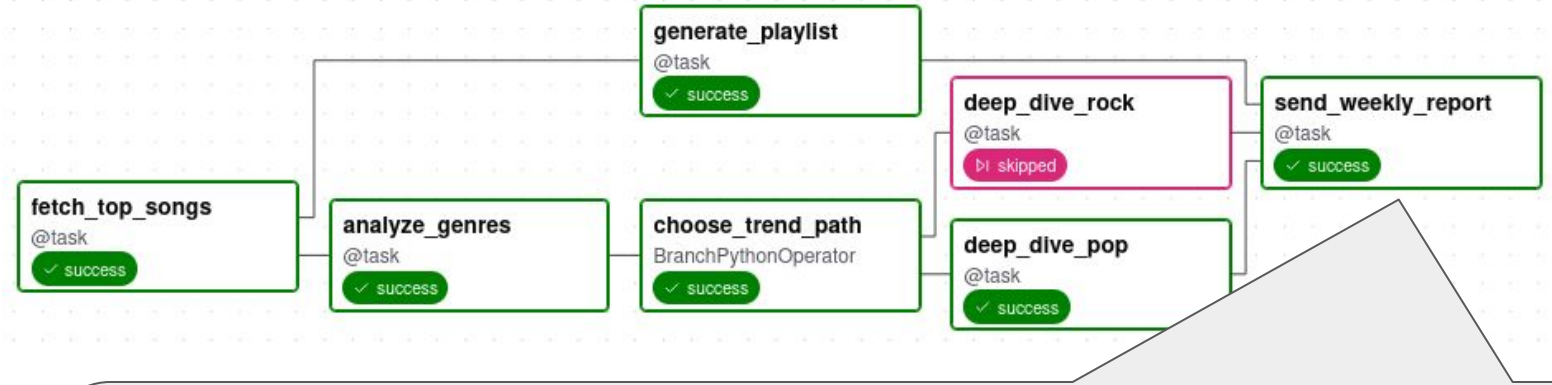






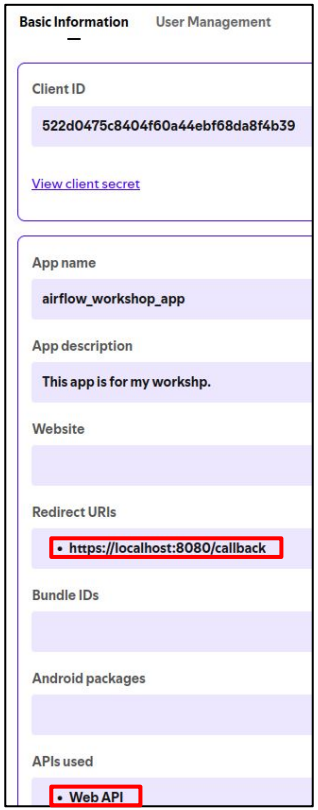
docker cp <container_id>:/tmp/airflow_weather/temperature_chart.png .
scp username@remote_host:/path/to/image.jpg /local/path/

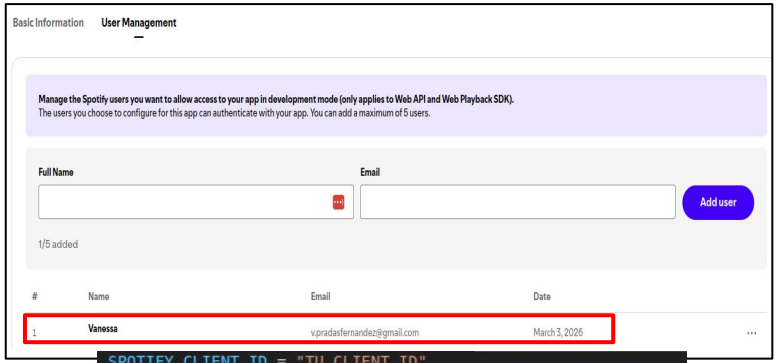
DAG de tendencias de Spotify




```
[2026-03-23, 10:09:40] INFO - Done. Returned value was: [{'title': 'Cruel Summer', 'artist': 'Taylor Swift', 'genre':
    'Pop', 'streams': 3199260}, {'title': 'Greedy', 'artist': 'Tate McRae', 'genre': 'Pop', 'streams': 2644981},
 {'title': 'Flowers', 'artist': 'Miley Cyrus', 'genre': 'Pop', 'streams': 2541995}, {'title': 'Kill Bill', 'artist':
  'SZA', 'genre': 'R&B', 'streams': 2405859}, {'title': 'Last Night', 'artist': 'Morgan Wallen', 'genre': 'Country',
'streams': 2390869}, {'title': 'Vampire', 'artist': 'Olivia Rodrigo', 'genre': 'Pop', 'streams': 2236957}, {'title':
   'Paint The Town Red', 'artist': 'Doja Cat', 'genre': 'Hip-Hop', 'streams': 1872379}, {'title': 'Barbie World',
   'artist': 'Nicki Minaj', 'genre': 'Hip-Hop', 'streams': 1722568}, {'title': 'Snooze', 'artist': 'SZA', 'genre':
 'R&B', 'streams': 1523806}, {'title': 'The Emptiness Machine', 'artist': 'Linkin Park', 'genre': 'Rock', 'streams':
 1440551}, {'title': 'Running Up That Hill', 'artist': 'Kate Bush', 'genre': 'Rock', 'streams': 1325446}, {'title':
                    'Lux Aeterna', 'artist': 'Metallica', 'genre': 'Rock', 'streams': 844900}]:
       source="airflow.task.operators.airflow.providers.standard.decorators.python._PythonDecoratedOperator"
```

DAG de comunicación entre tareas (XCom)



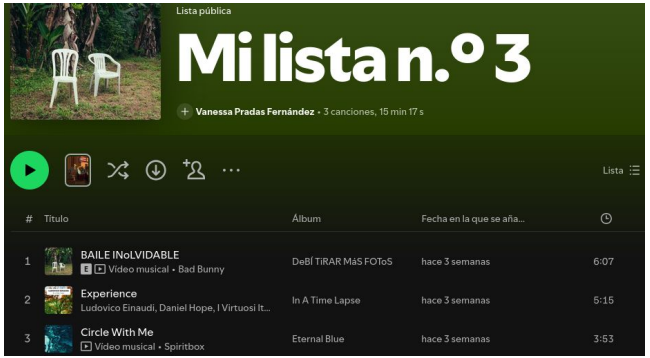


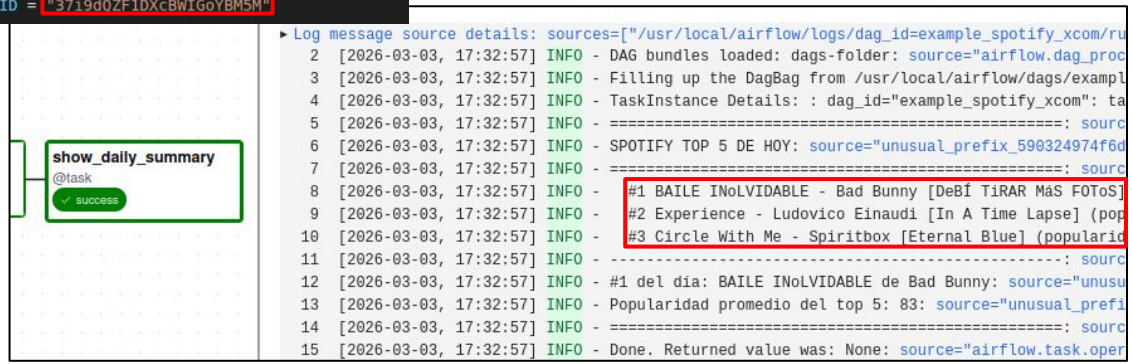

```
SPOTIFY_CLIENT_ID = "TU_CLIENT_ID"
SPOTIFY_CLIENT_SECRET = "TU_CLIENT_SECRET"

SPOTIFY_TOKEN_URL = "https://accounts.spotify.com/api/token"
SPOTIFY_API_BASE_URL = "https://api.spotify.com/v1"

# Tu playlist de Spotify
TOP_HITS_PLAYLIST_ID = "37i9dQZF1DXcBWIGoYBM5M"
```

Lista pública





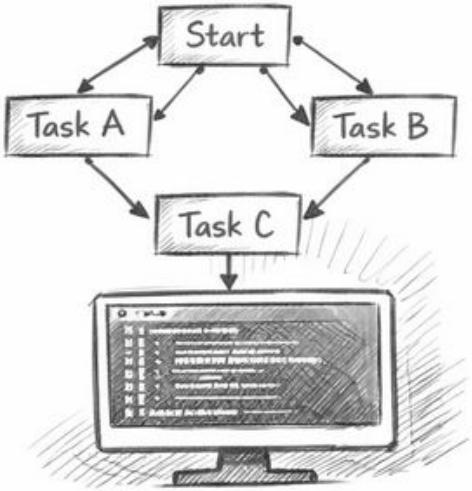


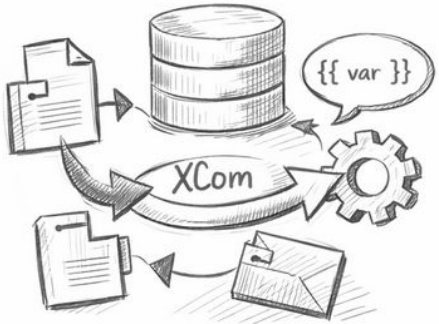
02_
Objetivos

Objetivos

1

Entender cómo se construye y ejecuta un DAG más avanzado





2

Entender cómo se comunican las tareas entre sí

3

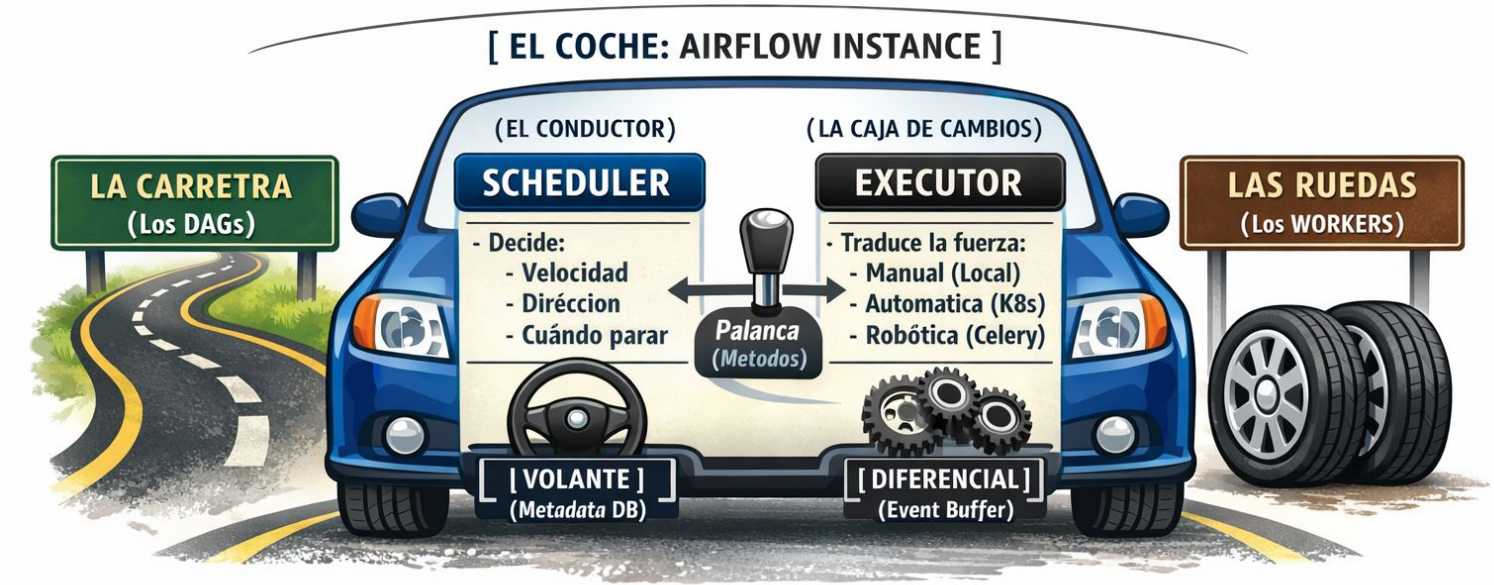
Aplicar buenas prácticas en la construcción de pipelines





03_
Conceptos avanzados

Conceptos avanzados - Analogía del coche



Conceptos avanzados - Flujo interno

Orden de pasos:

1. El **Scheduler** evalúa los DAGs y detecta el estado *scheduled*.
2. El **Executor** recibe las tareas y decide cómo se lanzan.
3. El **Worker** ejecuta el código del operador y reporta el estado (exitoso o fallido).

Scheduler → Executor → Worker

Conceptos avanzados - Scheduler

¿**Qué** tarea se ejecuta? ¿**Cuándo** se ejecuta? ¿Cómo se optimiza?



Scheduler

Configuración en el proyecto...

```python
@dag(
    dag_id="etl_employee",
    description="ETL de empleados",
    default_args=default_args,
    schedule="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["rrhh","employee"],
)
```

```python
default_args = {
    "owner": "Vanessa Pradas Fernandez"
    "depends_on_past": False,
    "email_on_failure": True,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}
```

Conceptos avanzados - Scheduler

En el archivo de configuración **airflow.cfg**...

```
[core]

# The amount of parallelism as a setting to the executor. This defines
# the max number of task instances that should run simultaneously
# on this airflow installation
parallelism = 32
```

```
# The maximum number of active DAG runs per DAG
max_active_runs_per_dag = 16
```



Scheduler

Conceptos avanzados - Scheduler

```
[scheduler]
# Monitoring
job_heartbeat_sec = 5
scheduler_heartbeat_sec = 5
scheduler_health_check_threshold = 30
scheduler_zombie_task_threshold = 300

# Process LifeCycle
run_duration = -1
num_runs = -1

# DAGs processing
min_file_process_interval = 0
processor_poll_interval = 1
dag_dir_list_interval = 300
max_threads = 2

# Execution
catchup_by_default = True
use_job_schedule = True
allow_trigger_in_future = False

# Performance and queries
print_stats_interval = 30
max_tis_per_query = 512
```

Monitoreo: se informa constantemente a la bbdd de la actividad (tareas y scheduler).

Ciclo de vida del proceso: limitar la duración de los procesos por tiempo o iteraciones.

Procesamiento de DAGs: se basa en la lectura del código (convertirlo en tareas ejecutables).

Rendimiento y queries: se regula la velocidad de la comunicación con la bbdd.



Scheduler

Conceptos avanzados - Executor

¿**Cómo** se ejecuta la tarea? ¿**Dónde** se ejecuta? ¿Tipos de Executor?



Executor

- Es instanciado por el Scheduler.
- Orquesta la ejecución.
- Se encarga de gestionar el paralelismo, el aislamiento y la distribución.

Conceptos avanzados - Executor

execute_async()

heartbeat()

sync()

get_event_buffer()

send_callback()

queue_workload()

has_task()

BaseExecutor: interfaz pública con la que el scheduler interactúa (es la clase abstracta de la que derivan todos los Executors).

https://airflow.apache.org/docs/apache-airflow/2.5.0/_api/airflow/executors/base_executor/index.html

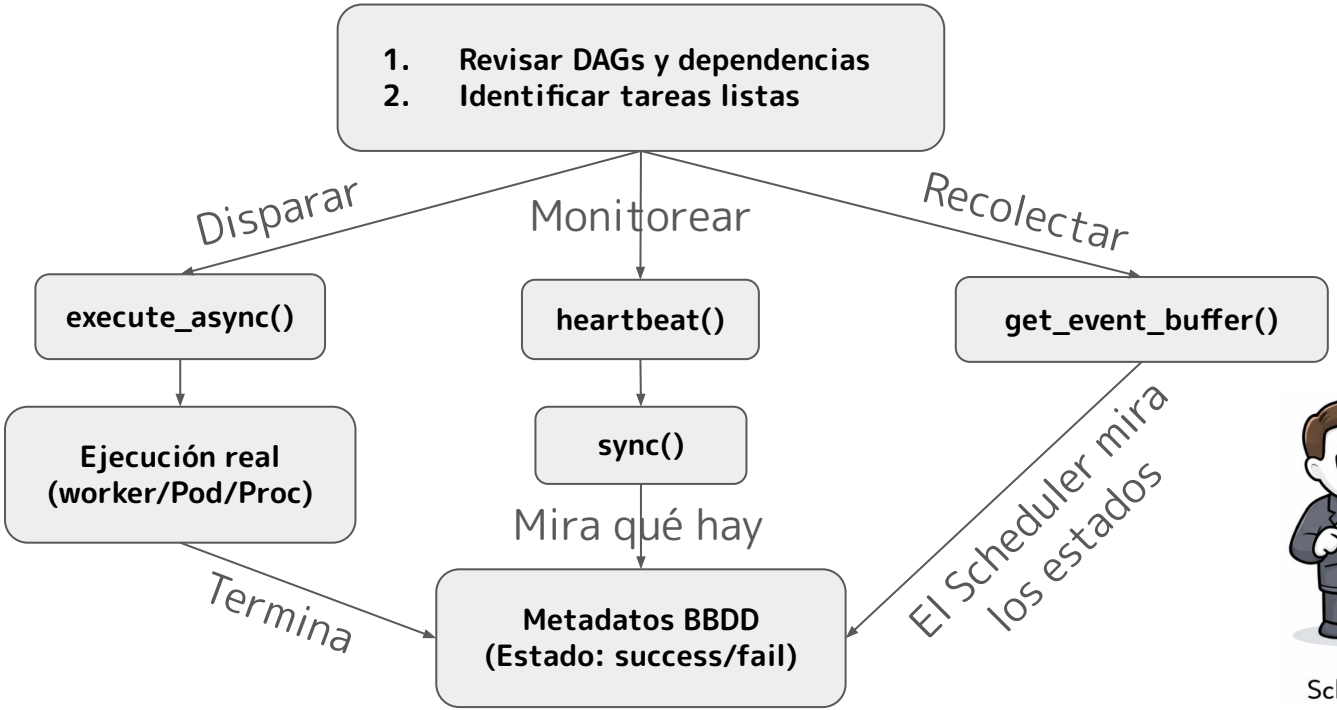
https://github.com/apache/airflow/blob/main/airflow-core/src/airflow/executors/base_executor.py

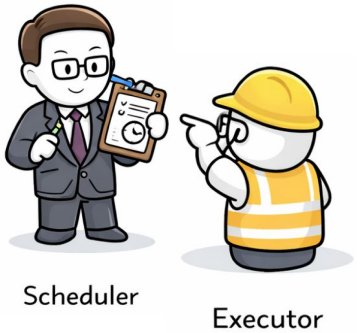


Executor

Conceptos avanzados - Executor

Ciclo de vida del Scheduler





Conceptos avanzados - Executor

Ciclo de vida del Executor (Bucle)

1º Ingestión

execute_async(task)
—
[Diccionario: QUEUED]

2º Latido (Heartbeat)

¿Hay slots libres?

3º Monitoreo

¿Cómo va el worker?

Fallo

¿Retry?

(Procesos / Pod / Mensaje)
—
[Estado: FAILED]

Retry

Éxito

4º Buffering

Mover a Event Buffer
—
[Event Buffer]

5º Reporting

get_event_buffer()
—
Actualizar Scheduler y limpiar memoria



Executor

Conceptos avanzados - Executor

Ejecutores locales (de un nodo)

SequentialExecutor: es síncrono y no soporta escrituras concurrentes (*unit_of_concurrency = 1*).

LocalExecutor: crea un subproceso por cada tarea y emplea colas con los IDs de las tareas. Escala verticalmente (limitado por las especificaciones de la máquina).



Executor

Conceptos avanzados - Executor

Ejecutores distribuidos (multi-nodo)

CeleryExecutor: necesita de un sistema de mensajería (como Redis) y una base de datos (como PostgreSQL) que guarde los metadatos de Airflow (estados, timestamps, logs…).

KubernetesExecutor: la orquestación es efímera porque no hay workers fijos, se crea un *Pod* por cada instancia de tarea.



Executor

Conceptos avanzados - Executor

Cuándo usar cada Executor

Executor	Características principales	Cuándo usarlo
SequentialExecutor	1 tarea a la vez Sin paralelismo	Desarrollo o pruebas
LocalExecutor	Multiprocessing Sin broker	Producción pequeño o mediano 1 sola máquina
CeleryExecutor	Celery + Broker Escalado horizontal	Producción grande Alta disponibilidad y escalabilidad
KubernetesExecutor	Cada tarea en un pod Escalado auto	Entornos cloud Aislamiento por tarea

Conceptos avanzados - Worker



Worker

- Es el componente que ejecuta las tareas.
- Transforma el estado de una instancia de tarea de RUNNING a SUCCESS o FAILED.
- A partir de Airflow 3.X, la comunicación se hace vía REST hacia el API Server, incluyendo heartbeats, actualización de estado y acceso a metadatos como Variables o Connections (según el backend de secretos configurado).
- Parsea el código y ejecuta los operadores (**operator.execute()**).

Conceptos avanzados - Worker

```
# Relevant worker parameters
parallelism = 32
worker_concurrency = 16
worker_prefetch_multiplier = 1
broker_url = redis://redis:6379/1
remote_logging = False
```



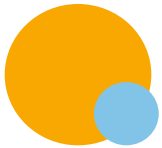
Worker

```
task_runner = StandardTaskRunner
```

Legacy, es de Airflow 2.X. Reemplazado por Task SDK

Conceptos avanzados - Airflow 3.X vs 2.X

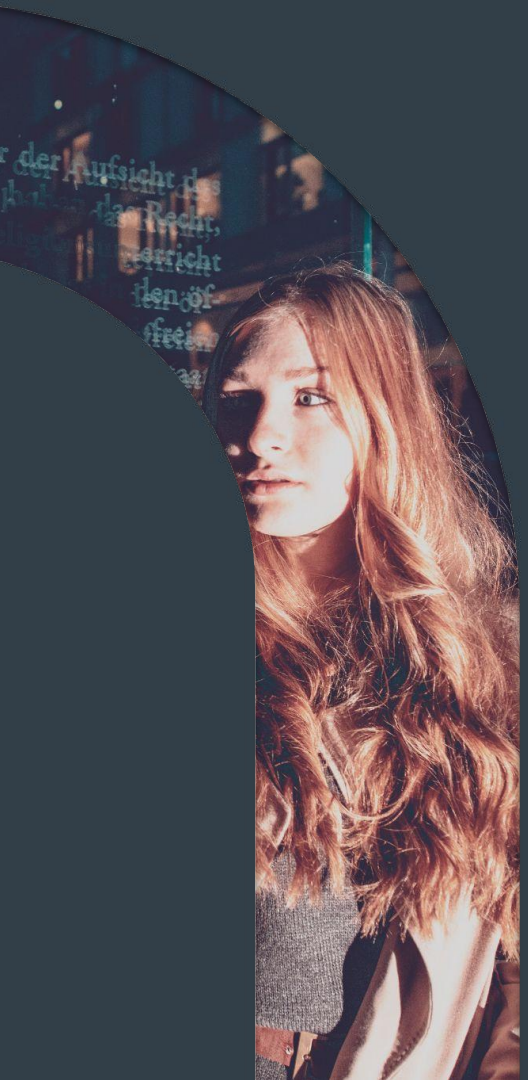
Característica	Airflow 2.X	Airflow 3.X
Comunicación del W	Directa con la BBDD	Mediante API Server
Ejecución de tareas	Paquete completo de Airflow	Task SDK
Escalabilidad	Limitada por conexiones a BBDD	(Virtualmente ilimitada) API Server
Logs	Escritura en un cloud storage	Stream hacia el API Server
Orquestación	Basada en tiempo (Cron)	Basada en *assets* (eventos)



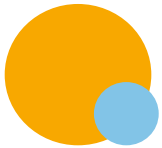
Conceptos avanzados - Otros



Concepto	Qué gestiona	Utilidad
Sensores	Flujo reactivo	Permite que el DAG espere eventos externos (archivos, SQL, APIs) mediante sondeo (polling).
Triggers	Eficiencia de recursos	Permite esperas asíncronas sin ocupar slots del worker, mejorando escalabilidad y costes.
XComs	Estado compartido	Intercambio de datos ligeros (metadatos, IDs) entre tareas independientes.
Estados	Ciclo de vida	Proporciona observabilidad total y permite la recuperación de fallos (retries y troubleshooting).
SLAs	Compromisos de tiempo	Monitoriza el cumplimiento de plazos de entrega y alerta de retrasos.



04_
Buenas prácticas



Buenas prácticas



- Diseñar DAGs pequeños y modulares
- Limitar la paralelización de las tareas
- Definir tareas idempotentes y atómicas
- Usar nombres claros y consistentes para DAGs y tareas.
- Escoger el operador adecuado a la tarea
- Buena gestión del tiempo de ejecución: *start_date* (schedule), *max_active_runs*, *concurrency* y *catchup*
- Habilitar logs para tener un histórico que facilite la depuración (en S3, GCS…)
- Configurar alertas y retries para tareas fallidas
- Versionar los DAGs (Git)

Gracias!

aulaupm@nextdigital.es

Orquesta tus datos, no tus dolores de cabeza

Anónimo

